

PROJECT WORK PLAN

From "Feature-Complete" to Shipped.

A grounded execution plan covering frontend verification, live backend deployment & integration, real-device QA, and launch readiness for the GetDraft sports-recruiting platform.

Where Talent Has No Borders.

PREPARED

June 5, 2026

REPOSITORY

moblie app · master

STACK

Expo SDK 54 · NestJS · Supabase

HORIZON

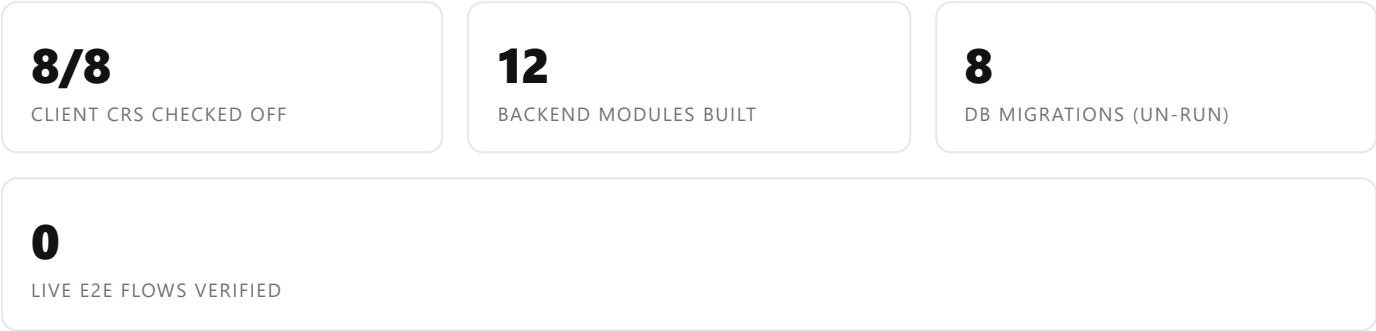
~5-6 weeks to demo-ready

Executive Summary

GetDraft's frontend and backend are both **built**: all eight client change requests (CRs) are checked off in the sprint plan, the React Native app ships every screen with a graceful mock-data fallback, and the NestJS backend has all 12 modules, 8 SQL migrations, and a test suite in place. The remaining work is **not "build features"** — it is **verify, connect, harden, and ship**.

This plan is grounded in a direct scan of the repository rather than the sprint checkboxes, which surfaced three things worth acting on immediately:

- **The backend is closer to live than the docs say.** The .env already holds real Supabase, Stripe, SMTP, Twilio (phone verification) and Didit (identity verification) credentials. Only SUPABASE_JWT_SECRET and EXPO_ACCESS_TOKEN are empty.
- **Scope has grown beyond the original Notion plan** — phone verification (Twilio) and KYC/identity verification (Didit) now exist and a migration 008 was added. These need their own verification and aren't reflected in the sprint doc.
- **The genuine gaps are integration and QA:** nobody has run the app against the live backend end-to-end, and device testing was done on the iOS simulator only — physical iOS and Android testing is outstanding.



How this plan is organized

- 1 Current State Assessment — what's real vs. assumed
- 2 Gap Analysis — the work that actually remains
- 3 Work Plan — four phases, task-by-task
- 4 Timeline & Critical Path
- 5 Risk Register
- 6 Definition of Done
- 7 Immediate Next Steps (this week)
- 8 Open Questions

1 • Current State Assessment

Status is based on a repository scan on June 5, 2026. "Confidence" reflects how strongly the code confirms the claim — high means verified by inspection, low means the checkbox says done but the code wasn't audited.

WORKSTREAM	STATUS	EVIDENCE IN REPO	CONFIDENCE
GetDraft - Project Work Plan			Confidential
Frontend — rebrand & copy (CR-4/5/6/8)	BUILT	Tabs incl. matches.tsx (Draft Board), brand copy in constants	Medium
Frontend — dark theme (CR-7)	BUILT	config/colors.ts black palette; splash on #121212	Medium
Frontend — 3D globe + welcome (CR-1/3)	BUILT	app/(tabs)/globe.tsx, WebView+Three.js, assets/globe.html	Medium
Frontend — stats animation (CR-2)	VERIFY	components/welcome/ has only WelcomeScreen + dots — StatsAnimationScreen not where doc says	Low
Frontend — API service layer	BUILT	13 files in services/ w/ JWT refresh + mock fallback	High
Backend — 12 modules	BUILT	backend/src/modules/ all present (auth...admin)	High
Backend — DB schema	UN-RUN	Migrations 001-008 exist; not yet applied to a project	High
Backend — credentials	PARTIAL	Supabase/Stripe/SMTP/Twilio/Didit set; JWT secret + Expo token empty	High
Backend — automated tests	PRESENT	5 unit spec suites + 1 e2e suite; e2e auto-skips w/o creds	High
Deployment (Railway)	NOT STARTED	No deploy config / live URL verified	High
Live FE↔BE integration	NOT STARTED	App still runs on mock fallback; no live run logged	High
Real-device QA (iOS/Android)	NOT STARTED	Sprint note: "tested on iOS simulator only"	High

Doc drift to fix: the sprint plan's Sprint 6 & 7 are labelled "IN PROGRESS" while every box under them is ticked; the CR callout says "7 change requests" but lists 8; and Twilio/Didit integrations aren't documented at all. The plan below treats the code as ground truth, not the checkboxes.

2 · Gap Analysis — What Actually Remains

Five buckets of work stand between the current state and a client-demoable, launch-track build.

G1 — Frontend verification & hardening

Trust the code, not the checkboxes. Audit each of the 8 CRs against its acceptance criteria, confirm the stats screen exists and is wired into the welcome flow, and confirm the globe holds its 60fps / <100MB budget on real hardware (covered in QA).

G2 — Backend go-live

Apply the 8 migrations to the Supabase project, fill the two empty secrets, verify all 12 modules boot, run the e2e suite against live creds (it currently auto-skips), and deploy to Railway with a stable public URL.

G3 — Live integration (remove the mock crutch)

Point the app at the deployed API, walk every role's flow against real data, and confirm the graceful fallbacks stay as a safety net but are no longer doing the heavy lifting. Validate the business rules (parent → 403 on discover, 11th basic swipe → 429, auto-downgrade on cancel).

G4 — Newly-discovered scope: Twilio + Ddidit

Phone verification and identity/KYC verification exist in the backend but aren't in the sprint plan. They need their own test pass, frontend wiring confirmation, and a documented decision on whether they're in-scope for the client demo.

G5 — Cross-platform QA & launch prep

Physical iOS + Android testing, performance budgets, EAS demo build, and store-readiness items (icons, privacy policy, listings) — plus reconciling the project docs with reality.

3 · Work Plan

Four phases. Phases A and B run largely in parallel (different surfaces), then converge at C. Estimates are working-hours for one engineer; priority drives ordering within a phase.

Priority: **HIGH** **MEDIUM** **LOW** · "Done when" is the acceptance bar for each task.

GetDraft · Project Work Plan

Confidentia

Phase A · Frontend Verification & Hardening

~16h · Week 1

ID	TASK	PRIORITY	EST	DONE WHEN
A1	Audit CR-1/2/3 (globe, stats, globe tab) against acceptance criteria; locate/confirm StatsAnimationScreen and welcome-flow wiring	HIGH	4h	Welcome → globe → stats → auth flow runs; each criterion ticked against real code
A2	Audit CR-4/5/6/8 (Draft Board, Draft/Pass, Game On!, CEGEP) for stray "Like/Match/Nope" copy & CEGEP ordering in all 15 sports	MEDIUM	3h	Grep finds zero legacy terms; CEGEP sits before U SPORTS everywhere
A3	Audit CR-7 dark theme — sweep every screen for "bright white" leaks; confirm dark forced as default	HIGH	3h	No #FFFFFF backgrounds; cards #1E1E1E; screenshots logged
A4	TypeScript + lint clean; fix any red squiggles introduced during rebrand	MEDIUM	2h	tsc --noEmit & eslint pass with 0 errors
A5	Confirm service layer reads API origin from config (not hardcoded localhost) for device builds	HIGH	2h	API base resolves from env/app config; works off-localhost
A6	Fix doc drift: TODO.md / sprint labels / CR count reconciled to reality	LOW	2h	Docs match code; Twilio/Didit documented

Phase B · Backend Go-Live

~20h · Weeks 1–2

ID	TASK	PRIORITY	EST	DONE WHEN
B1	Stand up Supabase project; run migrations 001–008; verify schema (14 tables, enums, RLS)	HIGH	4h	All migrations apply clean; RLS policies active; tables match plan
B2	Fill the two empty secrets — SUPABASE_JWT_SECRET, EXPO_ACCESS_TOKEN; confirm all set keys are valid (test mode where applicable)	HIGH	2h	Backend boots with no missing-config errors; keys validated
B3	Local boot of all 12 modules; smoke each module's primary endpoint via the executable API tests	HIGH	3h	App starts; health + auth + discover endpoints return 200
B4	Run e2e suite against live Supabase (it auto-skips without creds today)	HIGH	3h	E2E suite executes (not skipped) and passes
B5	Configure Stripe products/prices (basic \$0, starter \$3, pro \$7, premium \$15); wire price IDs; test checkout + webhook	HIGH	4h	Test-mode checkout completes; webhook updates subscription; downgrade-on-cancel works
B6	Deploy to Railway from backend/; set env; obtain stable public URL; enable CORS for app	HIGH	4h	Public URL responds; CORS allows app origin; logs clean

Phase C · Live Integration & Cross-Platform QA

~24h · Weeks 2–4

ID	TASK	PRIORITY	EST	DONE WHEN
C1	Point app at deployed API; run full flow per role (athlete, parent, coach, recruiter) against live data	HIGH	5h	Each role completes signup → core action chat on real data, no mock fallback hit
C2	Validate business rules: parent → 403 on discover; 11th basic swipe → 429; auto-downgrade on cancel	HIGH	3h	All three rules observed live with correct status codes/UX
C3	Verify Twilio phone verification & Didit identity verification end-to-end (newly-found scope)	MEDIUM	4h	Real phone OTP + ID check pass on device; failure UX is sane
C4	Real-time chat over Socket.IO against live backend (incl. on-device, backgrounded app)	HIGH	3h	Messages deliver <2s both directions; reconnect works
C5	Physical-device QA: iPhone 12+ & Pixel 6+ — globe 60fps / <100MB, cold-start <8s, push notifications	HIGH	5h	Budgets met on both platforms; push received on device
C6	Bug-fix buffer from C1–C5 findings	HIGH	4h	P0/P1 bugs closed; P2 triaged with owners

Phase D · Demo & Launch Readiness

~14h · Weeks 4–5

ID	TASK	PRIORITY	EST	DONE WHEN
D1	EAS demo build (internal distribution) for iOS + Android against live backend	HIGH	4h	Installable builds shared; launch clean on test devices
D2	Client demo dry-run with a scripted happy-path per role	HIGH	2h	Full demo runs without falling back to mock data
D3	Store-readiness checklist: app icons, splash, privacy policy, listing copy, screenshots	MEDIUM	4h	iOS + Play listings drafted; assets exported
D4	Observability: backend logging/alerting + crash reporting (Sentry) on the app	MEDIUM	3h	Errors visible in a dashboard; crash reports flow
D5	Handoff doc: runbook (deploy, rotate keys, restore), known issues, backlog	LOW	1h	A new dev can deploy + operate from the doc alone

Total: ~74 working-hours across four phases. With one engineer at a realistic pace that's ~5–6 calendar weeks; with frontend (A) and backend (B) split across two people, the critical path compresses to ~4 weeks.

4 · Timeline & Critical Path

Indicative 5-week schedule (single-engineer sequencing; A and B interleave). Bars show the active phase each week.

Phase	Wk 1	Wk 2	Wk 3	Wk 4	Wk 5
A · FE verification & hardening	A1–A6				
B · Backend go-live	B1–B3	B4–B6			
C · Live integration & QA		C1–C2	C3–C4	C5–C6	
D · Demo & launch readiness				D1–D2	D3–D5

Bars colored by phase. The hard dependency is **B6 (deploy) → C1 (live integration)**; everything in C is blocked until the backend has a public URL.

Critical path

B1 Supabase up → **B2** secrets → **B3** modules boot → **B6** Railway deploy → **C1** live integration → **C5** device QA → **C6** bug-fix → **D1** demo build → **D2** client demo.

Parallelizable: all of Phase A and the Stripe/Twilio/Didit verification (B5, C3) can proceed alongside the critical path. If two people are available, put one on A+frontend integration and one on B+infra.

5 · Risk Register

RISK	LIKELIHOOD	IMPACT	MITIGATION
Globe (WebView+Three.js) misses 60fps / <100MB on mid-range Android	MEDIUM	HIGH	Test early on Pixel 6+ (C5); have a static-globe fallback ready; cap continent light count
Requires native dev build — Three.js WebView crashes in Expo Go	HIGH	MED	Standardize on expo run: dev/EAS builds; document; never demo via Expo Go
Live data exposes RLS / business-rule gaps the mock layer hid	HIGH	HIGH	C1–C2 explicitly test each role + rule against live creds; e2e suite (B4) gates deploy
Stripe webhooks fail silently in production (downgrade not firing)	MEDIUM	HIGH	Verify webhook signature + endpoint post-deploy (B5/B6); log every subscription event (D4)
Twilio/Didit are real paid integrations with quotas & sandbox quirks	MEDIUM	MED	Use test phones/sandbox first (C3); confirm scope with client before live KYC
Secrets currently live in .env in the repo tree	HIGH	HIGH	Confirm .env is git-ignored; rotate any committed keys; move prod secrets to Railway vars
Mock-fallback masks integration failures (looks like it works, isn't)	MEDIUM	MED	Add a dev flag to disable fallback during C1–C4 so failures surface loudly

Act on the secrets risk first. Before anything else, confirm .env is not tracked by git and rotate keys if it ever was. (The repo's recent commits added a .gitignore entry for .env — verify it took effect and no keys are in history.)

6 · Definition of Done

The project is "demo-ready" when all of the following hold simultaneously:

- All 8 CRs verified against acceptance criteria **in code**, not just checked off — with screenshots.
- Backend deployed to a stable public URI; all 12 modules boot; e2e suite passes against live Supabase
- App runs end-to-end on the live API for all four roles with the mock fallback **disabled**.
- The three business rules (parent 403, basic-swipe 429, auto-downgrade) observed live.
- Stripe test-mode checkout + webhook + downgrade verified.
- Globe meets 60fps / <100MB / <8s cold-start on a physical iPhone 12+ **and** Pixel 6+.
- Push notifications received on a physical device for both platforms.
- EAS demo builds installable on iOS + Android; scripted demo runs without fallback.
- Secrets out of the repo and into Railway vars; .env git-ignored; keys rotated if exposed.
- Docs (TODO/sprint/handoff) reconciled with the shipped reality, incl. Twilio/Didit.

7 · Immediate Next Steps — This Week

Concrete, ordered, and small enough to start today. We can take these together one at a time.

#	ACTION	MAPS TO
1	Confirm .env is git-ignored & scan history for leaked keys; rotate if needed	Risk / B2
2	Run tsc --noEmit + eslint on the app and backend; capture the baseline error list	A4
3	Locate the stats-animation screen and trace the welcome → globe → stats → auth flow	A1
4	Stand up the Supabase project and run migrations 001–008	B1
5	Fill SUPABASE_JWT_SECRET + EXPO_ACCESS_TOKEN; boot the backend locally	B2/B3
6	Run the backend e2e suite against live creds and read the results together	B4

Suggested first move: steps 1–3 are pure frontend/safety and need no cloud setup — we can knock them out immediately while the Supabase project (step 4) spins up.

8 · Open Questions

- **Demo scope:** are Twilio phone-verification and Didit ID/KYC in-scope for the client demo, or deferred? (They exist in the backend but not the sprint plan.)
- **Hosting:** stay with Railway as planned, or has the target moved (Render/Fly/Supabase Edge)?
- **Pricing source of truth:** the sprint plan lists \$0/\$3/\$7/\$15 monthly; an older TODO lists annual \$550/\$250 figures. Which is current for Stripe products?
- **Coach role:** swipes on athletes per the plan, but no coach test credentials exist — is coach in-scope for this milestone?
- **Timeline:** is there a fixed client-demo date we're working back from?